Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет информационных технологий и программирования
Кафедра прикладной информатики

ДОМАШНЕЕ ЗАДАНИЕ №4

Технический дизайн микросервисной архитектуры

Выполнил: студент группы К3341
Кижваткин Н.М.

Санкт-Петербург
2025

1. ВВЕДЕНИЕ

Настоящий отчёт посвящён техническому дизайну микросервисной архитектуры системы Callmentor. Микросервисный подход позволяет разделить сложную систему на независимо развёртываемые компоненты, каждый из которых имеет чёткую зону ответственности. Это обеспечивает масштабируемость, отказоустойчивость и скорость разработки.

2. ОБОСНОВАНИЕ МИКРОСЕРВИСНОЙ АРХИТЕКТУРЫ

Callmentor является платформой обработки данных с принципиально разными вычислительными профилями компонентов:

- Gateway: высококонкурентная обработка HTTP-запросов, операции с БД — требует вертикального масштабирования.
- ML-ASR: вычислительно интенсивная транскрибация речи с использованием нейросетевых моделей — требует GPU и горизонтального масштабирования.
- ML-LLM: вызовы внешних LLM-API (OpenAI, Cloud.ru) с ожиданием — требует параллельной обработки очередей.
- Frontend: статический React-контент — требует CDN и отдельной оптимизации.

Разделение по микросервисам позволяет масштабировать и развёртывать каждый компонент независимо.

3. СОСТАВ МИКРОСЕРВИСОВ

Таблица 1 — Микросервисы системы Callmentor

Сервис	Технология	Порт	Назначение
nginx	Nginx	80 (1080 dev)	Reverse proxy, API-gateway
frontend	React/Vite	3000	SPA веб-интерфейс
gateway	FastAPI/Python	8000	REST API, бизнес-логика
gateway-db	PostgreSQL	5432	Хранение бизнес-данных
ml-service	FastAPI/Python	9001	Оркестратор ML-пайплайна
ml-asr-worker	FastStream/Python	8000 (internal)	ASR: транскрибация речи
ml-llm-worker	FastStream/Python	8000 (internal)	LLM: анализ транскрипций

ml-redis	Redis	5370	Кэш и состояние ML-задач
rabbitmq	RabbitMQ 3.13	5672/15672	Брокер сообщений
loki	Grafana Loki	3100	Агрегация логов
promtail	Grafana Promtail	-	Сборка логов Docker
prometheus	Prometheus	9090	Сбор метрик
node-exporter	Node Exporter	9100	Метрики хоста
cadvisor	cAdvisor	8081	Метрики контейнеров
grafana	Grafana	3000	Визуализация метрик и логов

4. ВЗАИМОДЕЙСТВИЕ СЕРВИСОВ

4.1. Синхронное взаимодействие (HTTP/REST)

Synchronous HTTP-взаимодействие используется между:

- Клиент (Browser) → Nginx → Gateway: все пользовательские запросы проходят через Nginx, который проксирует их на Gateway (/api/*) или Frontend (/).
- Gateway → ML-service: загрузка аудиофайла и создание задачи. Gateway вызывает ml-service по адресу PROCESS_AUDIO_URL (настраивается через переменную окружения).
- Gateway → ML-service: опрос статуса задачи через TASK_STATUS_URL.
- ML-service → Gateway: колбэк с результатами анализа на эндпоинт /ml-callback.

4.2. Асинхронное взаимодействие (RabbitMQ)

Асинхронное взаимодействие через RabbitMQ используется внутри ML-пайплайна. ML-service публикует сообщения в очереди, ASR-worker и LLM-worker их потребляют:

Таблица 2 — Очереди RabbitMQ

Очередь	Направление	Назначение
ASR_TRANSCRIPTION_QUEUE	ml-service → ml-asr	Стерео-аудио для прямой транскрибации
ASR_DIARIZATION_QUEUE	ml-service → ml-asr	Моно-аудио для диаризации + транскрибации
ASR_MANUAL_REVIEW_QUEUE	ml-asr → (parking)	Звонки с ошибками ASR на ручную проверку
LLM_TASK_QUEUE	ml-asr → ml-llm	Транскрипция для LLM-анализа
LLM_TASK_ERROR_QUEUE	ml-llm → (parking)	Ошибки LLM-анализа

LLM_POSTPROCESSING_QUEUE	ml-llm → ml-llm	Постобработка LLM-результатов
LLM_POSTPROCESSING_ERROR_QUEUE	(parking)	Ошибки постобработки
CALLBACK_QUEUE	ml-llm → ml-service	Результаты анализа для передачи в Gateway
CALLBACK_ERROR_QUEUE	(parking)	Ошибки колбэка
KPI_REPORT_QUEUE	gateway → ml-llm	Задачи еженедельного KPI-отчёта
OBJECTIONS_QUEUE	gateway → ml-llm	Задачи анализа возражений

5. ML-ПАЙПЛАЙН ОБРАБОТКИ ЗВОНКА

Полный пайплайн обработки одного звонка проходит следующие шаги:

1. Gateway принимает аудиофайл через POST /call/process.
2. Gateway передаёт файл в ML-service по HTTP.
3. ML-service проверяет количество каналов (pydub): если стерео с разными каналами → ASR_TRANSCRIPTION_QUEUE; если моно или стерео с идентичными каналами (корреляция > 0.999) → ASR_DIARIZATION_QUEUE.
4. ml-asr-worker выполняет транскрибацию: приоритетно использует Triton Inference Server (VastAI); при его недоступности — локальная модель T-one/GigaAM.
5. ml-asr-worker публикует транскрипцию в LLM_TASK_QUEUE.
6. ml-llm-worker загружает промты компании и оценивает транскрипцию: поддерживаемые провайдеры — OpenAI и Cloud.ru (SberBank GigaChat).
7. ml-llm-worker публикует результаты в CALLBACK_QUEUE.
8. ml-service получает результаты и отправляет колбэк в Gateway (POST /ml-callback).
9. Gateway сохраняет артефакты в БД, обновляет статус звонка на SUCCESS.

6. СИСТЕМА НАБЛЮДАЕМОСТИ (OBSERVABILITY)

Система оснащена полным стеком наблюдаемости на базе Grafana:

– Метрики: Prometheus собирает метрики с node-exporter (ресурсы хоста) и cAdvisor (метрики контейнеров Docker).

– Логи: Promtail собирает логи всех Docker-контейнеров и отправляет в Loki. Все сервисы пишут структурированные логи.

– Дашборды Grafana: 01-system-overview (общий обзор системы), 02-ml-pipeline (метрики ML-пайплайна), 03-rabbit-overview (метрики RabbitMQ), 04-node-monitoring (ресурсы сервера).

7. ВЫВОДЫ

Разработан технический дизайн микросервисной архитектуры системы Callmentor, включающей 16 сервисов. Архитектура обеспечивает: независимое масштабирование вычислительно интенсивных ML-компонентов, отказоустойчивость через очереди RabbitMQ с parking-очередями для ошибок, полную наблюдаемость через стек Prometheus-Loki-Grafana, горизонтальное масштабирование ML-воркеров.